GenAI Agent Governance Standards

> **Document Owner:** ML Engineering | **Status:** Approved | **Last Updated:** February 2026

Overview

Governance and lifecycle standards for GenAI agents on Databricks. This document covers the WHAT - requirements, thresholds, and compliance checkpoints.

For implementation patterns (code examples), see 51-genai-agent-patterns.md.

Golden Rules

ID	Rule	Severity	WAF Pillar
GS-01	Evaluate managed services before custom development	Critical	Governance
GS-02	Create evaluation dataset before development	Critical	Governance
GS-03	Pass LLM judge thresholds before production	Critical	Governance
GS-04	Enable production monitoring	Required	Governance
GS-05	Version prompts in Unity Catalog	Required	Governance
GS-06	Implement responsible AI practices for all production agents	Critical	Governance
GS-07	Integrate AI lifecycle with data lifecycle	Required	Governance

> **Cross-references:** Register models in Unity Catalog → see ML-07 (more specific).

Platform & Enterprise Prerequisites

> This document defines **solution patterns**. The following EA/PA rules are prerequisites.

Prerequisite	Rule IDs	Document
Models in Unity Catalog	ML-07	50-mlflow-model-patterns
UC governance	GOV-06	15-unity-catalog-governance
AI Gateway patterns	AG-01..05	53-ai-gateway-patterns

Managed Services Decision Framework

Always evaluate managed services FIRST. Document justification for custom development.

```
┌─────────────────────────────────────────────────────────────┐
│ Can Knowledge Assistant or Agent Bricks solve the problem? │
```

```
                    │
                    ├──────────────────────┐
                    ▼                      ▼
                ┌───┐                  ┌───┐
                │YES│                  │NO │
                └───┘                  └───┘
                    │                      │
                    ▼                      ▼
        Use Managed Service        Build Custom Agent
        (No custom code)           (Full compliance required)
```

Managed Service Capabilities

Service	Use Case	Capabilities
Knowledge Assistant	Document Q&A	Vector search, RAG, UC integration
Multi Agent Supervisor	Complex workflows	Agent orchestration, handoffs, routing
Genie Spaces	Data analytics	Natural language to SQL, semantic layer
AI Playground	Prototyping	Model comparison, prompt testing

Custom Development Justification

When choosing custom over managed, document:

1. Which managed services were evaluated
2. Why they don't meet requirements
3. Specific capability gaps
4. Maintenance ownership commitment

Evaluation Dataset Requirements

Create evaluation dataset BEFORE development begins.

Dataset Schema

Field	Type	Required	Description
`inputs`	dict[Any, Any]	Yes	Agent inputs (query, context)
`expectations.expected_response`	str	No	Ground truth answer
`expectations.expected_facts`	list[str]	No	Facts that must appear
`expectations.guidelines`	str	No	Rules agent must follow

Minimum Dataset Size

Environment	Minimum Records	Coverage
Development	50	Core use cases
Staging	200	Edge cases included
Production	500+	Full domain coverage

Dataset Creation

```
import mlflow.genai.datasets as datasets

eval_dataset = datasets.create_dataset(
    name="agent_eval_v1",
    source="catalog.schema.eval_records"
)
```

LLM Judge Thresholds

All thresholds MUST pass before production deployment.

Judge	Threshold	Blocking	Purpose
Safety	100%	Yes	No harmful content
Relevance	≥ 0.80	Yes	Query-response alignment
Groundedness	≥ 0.70	Yes	Factual accuracy
Guidelines	≥ 0.70	Yes	Policy compliance

Built-in Judges

Judge	Evaluates
`Relevance()`	Is response relevant to the query?
`Groundedness()`	Is response factually grounded in context?
`Safety()`	Does response contain harmful content?
`Guidelines(text)`	Does response follow specified guidelines?

Custom Judges

Create domain-specific judges for specialized evaluation:

```
from mlflow.genai.scorers import scorer

@scorer
```

```python
def sql_accuracy(outputs, expectations):
    """Verify generated SQL returns correct results."""
    generated = outputs.get("sql_query")
    expected = expectations.get("expected_sql")
    # Custom comparison logic
    return accuracy_score
```

Unity Catalog Model Registration

> **Note:** Model registration standards are defined in ML-07. The patterns below show GenAI-specific registration.

Naming Convention

```
{catalog}.{schema}.{agent_name}

Examples:
- prod_catalog.agents.data_platform_agent
- prod_catalog.agents.cost_analyzer_v2
```

Required Metadata

Metadata	Required	Example
Description	Yes	"Multi-domain data platform agent"
Tags	Yes	`domain:platform`, `team:ml-eng`
Aliases	Recommended	`Champion`, `Challenger`

Registration Benefits

- Centralized access control (GRANT/REVOKE)
- Version management with aliases
- Lineage tracking to training data
- Deployment governance
- Audit trail for compliance

Production Monitoring Requirements

Online Evaluation

Metric	Sample Rate	Alert Threshold
Safety	100%	Any failure
Relevance	10%	< 0.70 avg

Metric	Sample Rate	Alert Threshold
Latency	100%	P95 > 30s
Error Rate	100%	> 5%

Scorer Lifecycle

State	Description
Registered	Scorer defined but not running
Active	Evaluating traces at sample_rate
Stopped	sample_rate = 0, scorer preserved
Deleted	Permanently removed

Backfill Requirements

When deploying new scorers, backfill historical traces:

- Minimum: 30 days
- Recommended: 90 days

Memory & State Standards

Memory Architecture

Layer	Storage	TTL	Purpose
Short-term	CheckpointSaver	Session	Conversation context
Long-term	DatabricksStore	90 days	User preferences

Requirements

- Graceful degradation when memory unavailable
- No PII in long-term storage without encryption
- Session isolation (no cross-user leakage)

Prompt Management Standards

Storage

All prompts MUST be stored in Unity Catalog tables:

```
CREATE TABLE prod_catalog.prompts.agent_prompts (
    prompt_key STRING,
    prompt_text STRING,
```

```
    version INT,
    created_at TIMESTAMP,
    created_by STRING
);
```

Versioning

- Track all prompt changes with version numbers
- Log prompt versions used in each run
- Enable rollback to previous versions

Review Process

Change Type	Review Required
New prompt	Tech lead approval
Minor edit	Peer review
Major rewrite	Tech lead + evaluation

Compliance Checkpoints

Before Development

- [ ] Evaluated Knowledge Assistant for RAG use cases
- [ ] Evaluated Multi Agent Supervisor for workflows
- [ ] Evaluated Genie Spaces for analytics
- [ ] Documented justification for custom development
- [ ] Created evaluation dataset (min 50 records)
- [ ] Defined success criteria and thresholds

Before Staging

- [ ] Evaluation dataset expanded (min 200 records)
- [ ] All LLM judge thresholds passing
- [ ] MLflow Tracing enabled and verified
- [ ] Model registered in Unity Catalog
- [ ] Prompts stored in UC with versioning
- [ ] Memory fallback tested

Before Production

- [ ] Evaluation dataset at scale (min 500 records)
- [ ] Safety scoring at 100%
- [ ] Production monitoring configured
- [ ] Historical backfill completed (30+ days)
- [ ] Runbook documented
- [ ] On-call rotation assigned

- ☐ Rollback procedure tested

Responsible AI Practices

GS-06: All production agents must implement responsible AI guardrails.

Requirements

Practice	Implementation	Threshold
Safety filtering	AI Gateway guardrails or Llama Guard	100% of external-facing agents
PII protection	Input/output PII detection (Presidio)	Block SSN, credit cards; mask email, phone
Bias monitoring	Evaluate response fairness across demographics	Review quarterly
Transparency	Users informed they are interacting with AI	All external-facing agents
Human escalation	Ability to hand off to human for complex/sensitive queries	Required for customer-facing
Content attribution	Cite sources when grounding from documents	Required for RAG agents

Implementation Checklist

- ☐ AI Gateway guardrails enabled for external endpoints
- ☐ PII detection configured for input and output
- ☐ Safety scoring at 100% (GS-03)
- ☐ User disclosure ("I am an AI assistant") in system prompt
- ☐ Human escalation path documented and tested
- ☐ Bias review completed before launch

Anti-Patterns

- Deploying agents without safety filtering to external users
- Allowing agents to generate PII or credentials
- No human fallback for sensitive domains (HR, legal, compliance)

AI Lifecycle Integration with Data Lifecycle

GS-07: AI model development must align with the data platform lifecycle, reusing Gold layer assets and sharing feature tables between BI and AI workloads.

Why It Matters

- Prevents duplicate data pipelines for ML features vs BI metrics
- Ensures models train on the same governed, quality-checked data that powers dashboards

- Reduces time-to-model by leveraging existing Gold layer investments
- Creates a single source of truth for both analytical and predictive workloads

Data-to-AI Pipeline

```
Bronze → Silver → Gold (Dimensions & Facts)
                    │
                    ├── Semantic Layer (Metric Views, TVFs) → BI / Genie
                    │
                    └── Feature Tables (Unity Catalog) → ML Training →
Model Serving
```

Integration Patterns

Pattern	Description
Gold-to-Feature	Derive ML feature tables from Gold layer dimensions/facts
Shared Metrics	Reuse Metric View definitions as ML features where applicable
Evaluation on Gold	Use Gold layer data for model evaluation datasets
Prediction-to-Gold	Write model predictions back to Gold layer for BI consumption
Lineage Tracking	UC lineage traces features back through Gold → Silver → Bronze

Implementation

```python
# Feature table derived from Gold layer
from databricks.feature_engineering import FeatureEngineeringClient

fe = FeatureEngineeringClient()

# Create feature table from existing Gold dimension
feature_df = spark.sql("""
    SELECT
        customer_key,
        lifetime_value,
        order_count,
        avg_order_value,
        days_since_last_order,
        customer_segment
    FROM gold.dim_customer
    WHERE is_current = true
""")

fe.create_table(
    name="catalog.features.customer_features",
    primary_keys=["customer_key"],
    df=feature_df,
```

```
    description="Customer features derived from gold.dim_customer"
)
```

Exception Process

For threshold exceptions:

1. Document specific use case requiring exception
2. Provide alternative mitigation (e.g., human review)
3. Set review date (max 90 days)
4. Obtain VP-level approval

References

- Knowledge Assistant
- Agent Bricks
- Evaluation Datasets
- Production Quality Monitoring
- MLflow Tracing